5. Hafta

React II: Hook'lar, Form, Durum Paylaşımı

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- `useEffect`'i yan etki kavramı, bağımlılık dizisi ve temizleme fonksiyonuyla açıklayarak belge başlığını bir değere göre güncelleyen ve `setInterval` tabanlı, temizleme fonksiyonuyla düzgün sonlanan bir geri sayım örneğini uygulayabileceksiniz;
- `value`/`onChange` ile kontrollü girdi kuran, tiplenmiş bir form state'i (`select`/`checkbox` dahil) tutan, temel doğrulama yapıp hata mesajı gösteren ve `onSubmit`+ `preventDefault` kullanan bir form bileşeni (`BiletFormu`) oluşturabileceksiniz;
- Bir bileşen içinde hapis kalan durumu bir üst bileşene taşımanın (durum kaldırma) ve callback prop'larla yukarı bildirmenin sınırlarını (prop drilling sorunu) açıklayarak, `createContext`/ `useContext` ile bir Context (`SepetContext`) kurup birden fazla bileşenin aynı durumu (seçili koltuklar, toplam tutar) doğrudan okuyup güncellemesini sağlayabileceksiniz;
- Tekrar eden state/etki mantığını bir özel hook'a (`useSepet`, `useGeriSayim`) çıkararak bileşenlerden ayırabilecek, `children` ile bileşen kompozisyonunu (ve `render props` kalıbının adını) tanıyarak bir bileşeni ne zaman bölmeniz gerektiğine karar verebileceksiniz;
- Bu becerileri birleştirip Hafta 4'ün `Koltuk` içinde hapis kalan koltuk seçimini `SepetContext` üzerinden `App` düzeyine taşıyarak, `BiletFormu` gönderiminde sepet ve form verisinden `Bilet[]` nesneleri oluşturup ilgili koltukları `'dolu'`ya çevirerek Etkinlik/Bilet Sistemine ilk gerçek sepet katmanını ekleyebileceksiniz.

Hafta 4'ün sonunda `Koltuk` bileşeni kendi seçim durumunu (`useState`) tutuyordu, ama bu bilgi `Koltuk`'un içinde hapis kalıyordu — "`App`'te toplam kaç koltuk seçili, toplam tutar ne olduğunu nasıl görürüz?" sorusu bilinçli olarak yanıtsız bırakılmıştı. Bu hafta tam olarak bu soruyla açılır. Dört ünite boyunca sırasıyla bir bileşenin render'ın dışında bir işe (belge başlığı, bir zamanlayıcı) nasıl karışacağını (`useEffect`, Ünite 1), bir formun her tuş vuruşunu React'in state sistemine nasıl bağlayacağını (kontrollü girdi, Ünite 2), birden fazla bileşenin aynı durumu (sepet)

nasıl paylaşacağını (durum kaldırma + Context, Ünite 3) ve tekrar eden state/etki mantığının nasıl tek bir yerde (özel hook) toplanacağını (Ünite 4) göreceksiniz. Sunucu (FastAPI) bu hafta da yoktur — veri hâlâ Hafta 3'ün `veri.ts`'inden gelen sabit (mock) dizilerdir; React Router da bu hafta yoktur, tek bir `App` bileşeni sürüyor. Son ünitenin sonunda Etkinlik/Bilet Sisteminin ön yüzü, koltuk seçiminden bilet oluşumuna kadar ilk gerçek sepet katmanına kavuşacaktır.

5.1 `useEffect` ve Yaşam Döngüsü

1 • Ön-öğretim — Neden Önemli?

Geçen hafta `Koltuk` bileşeni kendi durumunu (`useState`) tutuyordu, ama sayfanın kendisi — sekme başlığı, bir geri sayım gibi "render'ın dışındaki" şeyler — hiç güncellenmiyordu. Bir bileşenin render işi dışında bir şey yapması (belge başlığını değiştirmek, bir zamanlayıcı kurmak) gerektiğinde `useState` tek başına yetmez — bu ünite bu "render dışı iş"i yönetmenin standart yolunu tanıtır.

1 • Ön-öğretim — Ön Bilgi

Hafta 4 Ünite 3'ün `useState`/değişmezlik ilkesini hatırlayın. Hafta 3 Ünite 4'ün `setTimeout`/olay döngüsü bilgisini de hatırlayın — bu ünitede bu bilgi `setInterval`'e genişleyecek. Son olarak YG II'nin `std::thread`'i başlatıp durdurma disiplinini hatırlayın: bir iş parçacığını `join`/`detach` etmeden bırakmak nasıl soruna yol açıyorsa, bu ünitede kuracağınız bir zamanlayıcıyı da durdurmayı unutmak aynı türden bir soruna yol açar.

Ünite 3'te (Hafta 4) `useState` ile bir bileşenin kendi hafızasını tuttuğunu gördünüz. Ama bazen bir bileşenin yapması gereken iş, render'ın — yani JSX üretmenin — sınırlarının dışına taşar: sekme başlığını değiştirmek, bir zamanlayıcı kurmak, (ileride Hf.7'de) bir ağ isteği göndermek. Bu ünite React'in bu "dış dünyaya dokunan" işleri yönetmenin standart yolunu — `useEffect`'i — tanıtır.

5.1.1 Yan etki: render işinin dışında kalan iş

Bir bileşen fonksiyonunun asıl görevi bellidir: mevcut `props`/ `state`'e bakıp bir JSX ağacı döndürmek. Ama bazen bir bileşenin yapması gereken iş bundan FAZLASIdır — sayfanın sekme başlığını değiştirmek, bir zamanlayıcı başlatmak, bir bağlantı açmak gibi, JSX üretmenin dışında kalan, "dış dünyaya" dokunan bir iş. Buna **yan etki** denir.

yan etki: Bir bileşenin render işinin dışında, dış dünyaya (belge, zamanlayıcı, ağ) dokunan iş (*side effect*).

Bu işi doğrudan bileşen gövdesine — ör. `document.title = ...` satırını render sırasında çalışacak bir yere — yazmak neden sorunludur? Çünkü React bir bileşeni istediği zaman, istediği kadar render edebilir (ör. geliştirme modunda `StrictMode` hataları erken yakalamak için bazı bileşenleri bilerek iki kez çağırır). Render sırasında doğrudan bir yan etki çalıştırmak, bu öngörülemez tekrar sayısı yüzünden aynı işin gereksiz yere birden çok kez — ya da yanlış zamanda — çalışmasına yol açar.

Analoji

Yan etkiyi bir dükkânın vitrin tabelasını, dükkânın içindeki stok değiştiğinde ayrıca güncellemesine benzetin: tezgâhtaki satış işlemi (render) bir şeydir, vitrindeki tabelayı değiştirmek (belge başlığı, bir zamanlayıcı) ise ayrı, ek bir iştir — biri diğerinin doğal, otomatik parçası değildir, bilinçli olarak yapılması gerekir.

5.1.2 `useEffect`: render bittikten sonra çalışan kod

React bu ihtiyaç için `useEffect` adlı bir Hook sunar. Bir bileşenin içinde şöyle çağrılır:

```
useEffect(() => {
  // yan etki burada çalışır
}, [bagimliliklar]);
```

İlk argüman bir FONKSİYONdur ve render tamamlandıktan sonra çalışır — React önce JSX'i ekrana uygular, ancak ondan sonra bu fonksiyonu çağırır. İkinci argüman ise bir **bağımlılık dizisi**dir — bu dizi, efekt fonksiyonunun ne zaman tekrar çalışacağına karar verir.

bağımlılık dizisi: `useEffect`'in ikinci argümanı; efektin ne zaman tekrar çalışacağını belirleyen değer listesi (*dependency array*).

Bağımlılık dizisinin üç farklı hâli, üç farklı davranış üretir:

- İkinci argüman hiç verilmezse, efekt her render'dan sonra çalışır.
- Bir **boş dizi** (`[]`) verilirse, efekt yalnızca ilk render'dan (*mount*) sonra bir kez çalışır.
- **Belirli değerlerle** (`[deger]`) verilirse, efekt yalnızca o değer(ler) bir önceki render'a göre değiştiyse tekrar çalışır.

Analoji

Bağımlılık dizisini bir alarmı "şu koşullardan biri değişirse çal" diye kurmaya benzetin: liste boş ise alarm yalnız kurulduğunda bir kez çalar; listede bir şey varsa yalnız O şey değiştiğinde çalar; liste hiç verilmezse alarm her olayda (her render'da) çalar.

Ortak Hata / Tuzak

Bağımlılık dizisini tamamen atlayıp efekt içinde bir `setState` çağırmak sonsuz döngüye yol açabilir: efekt her render'dan sonra çalışır, içindeki `setState` yeni bir render'ı tetikler, o render efekti tekrar çalıştırır, bu da yeni bir `setState` tetikler — döngü hiç durmaz. Bu yüzden bir efekt içinde state güncelliyorsanız, bağımlılık dizisini — efektin gerçekten hangi değerlere bağlı olduğunu düşünerek — mutlaka doğru belirlemeniz gerekir.

5.1.3 Örnek: sekme başlığını güncelleme

`AppIcerik` bileşeninde, sepetteki seçili koltuk sayısına göre tarayıcı sekmesinin başlığını güncelleyen bir `useEffect` şöyle yazılır:

```
1 useEffect(() => {
2   document.title =
```

```
    secilenKoltuklar.length > 0
      ? `(${secilenKoltuklar.length}) KTÜ Etkinlik ve Bilet Sistemi`
      : 'KTÜ Etkinlik ve Bilet Sistemi';
}, [secilenKoltuklar.length]);
```

Bağımlılık dizisi `[secilenKoltuklar.length]`'dir — yani efekt yalnızca seçili koltuk sayısı bir önceki render'a göre değiştiğinde tekrar çalışır, her render'da değil. `secilenKoltuklar` değişkeninin nereden geldiğini şimdilik merak etmeyin — bu ünite yalnızca `useEffect`'in kendisine, biçimine ve bağımlılık dizisine odaklanıyor; `secilenKoltuklar`'ın kaynağını (bir Context) Ünite 3'te göreceksiniz.

5.1.4 Temizleme fonksiyonu: bir sonraki kuruluma yer açmak

Bazı yan etkiler — bir zamanlayıcı kurmak, bir bağlantı açmak gibi — arkalarında bir "kaynak" bırakır; bu kaynağın bir noktada kapatılması/durdurulması gerekir. Bunun için efekt fonksiyonu, isteğe bağlı olarak bir başka fonksiyon döndürebilir; buna **temizleme fonksiyonu** denir. React bu fonksiyonu iki durumda çağırır: (a) bileşen ekrandan kaldırılmadan (*unmount*) hemen önce, ve (b) aynı efekt bağımlılık değişikliği yüzünden tekrar çalışmadan hemen önce — "bir önceki kurulumu temizleyip yeni kuruluma yer açma" mantığıyla.

temizleme fonksiyonu: Bir efektin geri döndürdüğü, bir önceki kurulumu (zamanlayıcı, bağlantı) sonlandıran fonksiyon (*cleanup function*).

Analoji

Temizleme fonksiyonunu bir toplantı odasını kullandıktan sonra ışığı söndürüp kapıyı kilitlemeye benzetin: bir sonraki kullanım (yeni render, ya da odanın tamamen boşaltılması/*unmount*) için ortamı temiz bırakırsınız; aksi hâlde ışık (zamanlayıcı) boş odada yanık kalır.

Şekil 5.1, bir bileşenin sahneye girmesinden (*mount*) sahneden kalkmasına (*unmount*) kadar bu sürecin tamamını bir zaman çizgisi olarak gösterir: mount → efekt çalışır → bağımlılık değişir → önce temizleme, sonra yeni efekt → unmount → son temizleme.

5.1.5 Örnek: `setInterval` ile geri sayım

Şimdi hem bir efekt kurmayı hem de onu düzgün temizlemeyi tek bir örnekte birleştirin. Aşağıdaki fonksiyon — bu ünitede henüz "özel hook" olarak adlandırılmaz, yalnızca "state ve efekt kullanan bir fonksiyon" olarak düşünün, resmî tanımını Ünite 4'te göreceksiniz — `useState` ile kalan saniyeyi tutar, `useEffect` içinde `setInterval` her saniye bu değeri günceller:

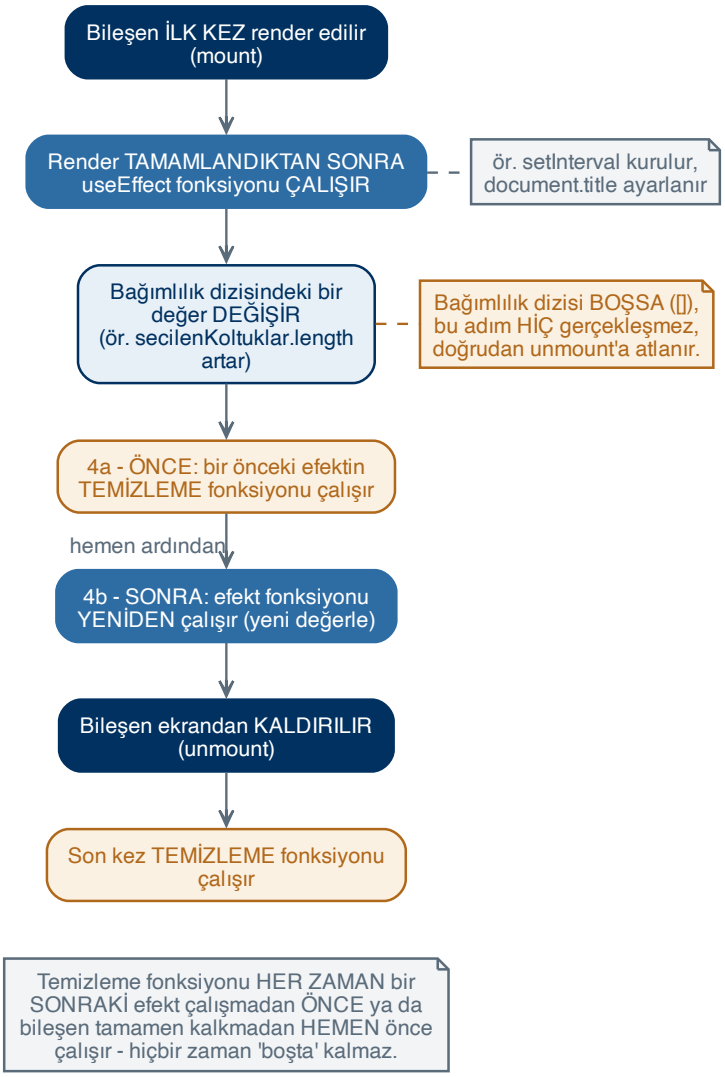
```
export function useGeriSayim(baslangicSaniye: number) {
  const [kalanSaniye, setKalanSaniye] = useState(baslangicSaniye);
  const bittiMi = kalanSaniye <= 0;

  useEffect(() => {
    if (bittiMi) return;

    const id = setInterval(() => {
      setKalanSaniye((onceki) => (onceki <= 1 ? 0 : onceki - 1));
    }, 1000);

    return () => clearInterval(id);
  }, [bittiMi]);

```



Şekil 5.1: Bir bileşenin *mount*'undan *unmount*'una kadar `useEffect`'in çalışma zamanları: efekt, bağımlılık değişince önce temizleme sonra yeniden çalışma, son olarak son temizleme.

```
  return { kalanSaniye, bittiMi };
}
```

`bittiMi true` olduğunda efekt erken `return` yapar — yani süre bittiğinde yeni bir zamanlayıcı kurulmaz. Efekt her çalıştığında (bağımlılık `bittiMi` değiştiğinde ya da bileşen kalktığında) döndürdüğü `() => clearInterval(id)` fonksiyonu önce çağrılır; bu, bir önceki `setInterval` çağrısını durdurur.

Ortak Hata / Tuzak

Bu `return () => clearInterval(id)` satırı olmasaydı ne olurdu? Bileşen ekrandan kalksa (*unmount*) bile zamanlayıcı arka planda çalışmaya devam ederdi — her saniye artık var olmayan bir bileşenin state'ini güncellemeye çalışırdı, bu da tarayıcı konsoluna bir uyarı basar ve bir bellek sızıntısına yol açar. YG II'de bir `std::thread`'i `join`/`detach` etmeden bırakmanın yol açtığı soruna — iş parçacığının programın geri kalanından bağımsız, kontrolsüz çalışmaya devam etmesine — benzer bir durumdur.

İpucu

Bir API'den veri çekmek de tipik bir yan etkidir, bu yüzden genellikle `useEffect` içinde yapılır — Hf.7'de `fetch` ile bunu göreceksiniz. Ama `fetch`, yükleme/hata/boş durumları ve CORS bu haftanın konusu değildir; bu haftanın iki `useEffect` örneği (başlık, geri sayım) hiçbir ağ isteği içermez.

3 · Katılım — Şimdi Siz Deneyin

`AppIcerik`'teki belge başlığı efektinin bağımlılık dizisini geçici olarak `[]` yapın (boş dizi); birkaç koltuk seçip sekme başlığının hâlâ güncellenip güncellenmediğini gözlemleyin. Sonra bağımlılık dizisini `[secilenKoltuklar.length]`'e geri döndürün.

Geribildirim. Bağımlılık dizisi `[]` olduğunda efekt yalnızca mount'ta bir kez çalışır; koltuk sayısı sonradan değişse de efekt bir daha çalışmaz, sekme başlığı ilk hâlinde donar. React, bir bağımlılık dizisi vermediğiniz sürece "ne zaman yeniden çalışacağını" sizden öğrenir, kendiliğinden tahmin etmez — bu yüzden bağımlılık dizisini doğru belirlemek her zaman sizin sorumluluğunuzdadır.

Kendinizi Sınayın 5.1

`useEffect(() => { ... }, [])` ile `useEffect(() => { ... })` (ikinci argüman tamamen atlanmış) arasındaki fark nedir; ikinci biçimde bir sonsuz döngü riski nasıl oluşabilir?

5 · İzleme — Aktarım — Başka Nerede?

Bu haftadan sonra "sayfa dışına açılan" her iş — belge başlığı, zamanlayıcı, Hf.7'de gerçek veri çekme, Hf.13'te bir WebSocket bağlantısı açıp kapatma — `useEffect` deseniyle yönetilecek; bugün öğrendiğiniz temizleme fonksiyonu, Hf.13'ün bağlantı kapatma ihtiyacında aynen karşınıza çıkacak.

5.2 Formlar ve Kontrollü Girdi

1 • Ön-öğretim — Neden Önemli?

Hafta 3'ün bilet formu düz HTML'di — `submit` olayında yalnızca `preventDefault()` çağrılıp konsola bir özet basılıyordu, form React'in ise haberi bile yoktu. Bu formu, doğrulama yapabileceğiniz, gönderim anında sepetle birleştirebileceğiniz bir React bileşenine dönüştürmezseniz, "ad boş mu", "koltuk seçildi mi" gibi basit bir kontrolü bile yapamazsınız.

1 • Ön-öğretim — Ön Bilgi

Hafta 2 Ünite 1'in `bilet_formu` HTML yapısını hatırlayın — aynı alanlar şimdi React'e taşınıyor. Hafta 4 Ünite 3'ün `useState` ve tiplenmiş olay işleyicisi bilgisini de hatırlayın — `onChange` da tıpkı `onClick` gibi bir JSX olay özniteliğidir.

Ünite 1'de bir bileşenin render dışına nasıl çıktığını gördünüz. Bu ünite ise render'ın tam içinde kalan, ama Hafta 3'te React'in hiç haberi olmadığı bir alanı — formları — ele alır: kullanıcının her tuş vuruşunu React'in `state` sistemine bağlamayı, doğrulama yapmayı ve bir form gönderimini React'in kendi mekanizmalarıyla yönetmeyi öğretir.

5.2.1 Kontrolsüz ve kontrollü girdi

kontrolsüz girdi: Değerini kendi DOM'unda tutan, React'in doğrudan izlemediği form öğesi (*uncontrolled input*).

Hafta 3'ün HTML formunda bir `<input>` kendi değerini tarayıcının DOM'unda tutuyordu — siz bir tuşa bastığınızda React'in bundan haberi yoktu, değeri yalnızca gönderim anında (`FormData` ile) okuyordunuz. Buna **kontrolsüz girdi** denir — "kontrolsüz", React'in bu değeri hiç kontrol etmediği, yalnızca tarayıcının kendi başına yönettiği anlamına gelir.

Buna karşılık, bu haftanın kullanacağı yaklaşımda bir **kontrollü girdi**nin değeri her zaman React `state`'inden okunur: kullanıcı bir tuşa bastığında `onChange` tetiklenir, bu olay işleyicisi `state`'i günceller, React yeniden render eder ve girdinin `value`'su güncel `state`'i yansıtır. Burada `state`, "tek doğruluk kaynağı" (*single source of truth*) hâline gelir — DOM'un kendi başına tuttuğu bir değer yoktur, DOM her zaman `state`'in bir YANSIMASIdır.

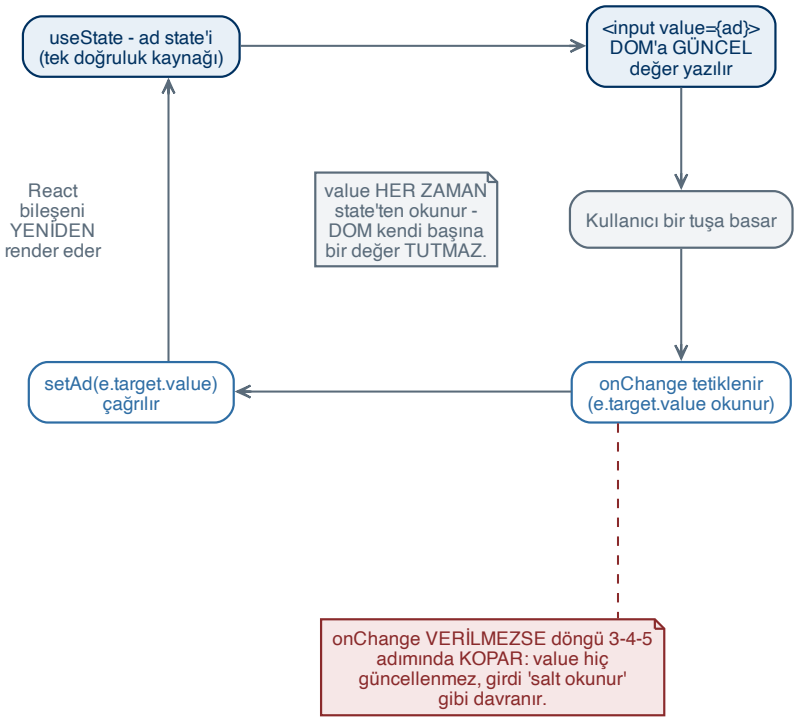
Analoji

Kontrollü girdiyi bir gişe memurunun, doldurduğunuz her formu anında bir deftere kopyalamasına benzetin: kâğıttaki (DOM'daki) yazı ile defterdeki (`state`'teki) kayıt her zaman aynı tutulur; memur (React) bu eşitliği her tuş vuruşunda `onChange` ile sağlar.

Şekil 5.2 bu döngüyü gösterir: `value` state'ten okunup DOM'a yazılır, kullanıcı yazar, `onChange` tetiklenir, `setState` çağrılır, bileşen yeniden render edilir ve güncel `value` tekrar DOM'a yazılır — kapalı bir döngüdür.

Ortak Hata / Tuzak

Bir `<input>`'a `value` verip `onChange` vermemek yaygın bir hatadır — React konsola bir uyarı basar ve kullanıcı girdiye hiçbir şey yazamaz. Neden? Çün-



Şekil 5.2: Kontrollü girdi döngüsü: `value` state'ten okunur, kullanıcı yazar, `onChange` tetiklenir, `setState` çağrılır, bileşen yeniden render eder, güncel `value` DOM'a yazılır.

kü `value` her render'da aynı `state`'ten okunur; kullanıcı bir harf yazsa bile `state`'i güncelleyecek tek yol (`onChange`) olmadığından, DOM'un kendi iç değeri her render'da eski `state`'e geri zorlanır — girdi âdeta "salt okunur" hâle gelir.

5.2.2 Tiplenmiş form state'i

form state'i: Bir formun tüm alanlarını tek bir tipte gruplayan durum nesnesi (*form state*).

Bir formda birden fazla alan varsa, her biri için ayrı bir `useState` çağırmak yerine, tüm alanları tek bir **form state'i**nde gruplarsınız. Ad ve koltuk sınıfını içeren `BiletFormVeri` arayüzü şöyle tanımlanır:

```
export interface BiletFormVeri {
  ad: string;
  koltukSinifi: 'standart' | 'vip';
  kampanyaOnay: boolean;
}

const BOS_FORM: BiletFormVeri = {
  ad: '',
  koltukSinifi: 'standart',
  kampanyaOnay: false,
};

interface BiletFormuProps {
  onBiletlerOlustu: (biletler: Bilet[]) => void;
}
```

Formun `useState` çağrısı ve sepetten okunan bilgiler — `form` ve `hata` state'leri, sepetten gelen `secilenKoltuklar`/ `sepetiTemizle` (bunların Context'ten nasıl geldiğini Ünite 3-4'te göreceksiniz) — şöyle bir araya gelir:

```
const [form, setForm] = useState<BiletFormVeri>(BOS_FORM);
const [hata, setHata] = useState<string | null>(null);
const { secilenKoltuklar, sepetiTemizle } = useSepet();
```

Bir alanı güncellerken diğer alanları korumak — Hafta 4 Ünite 3'ün değişmezlik ilkesinin form state'ine uygulanışıdır — için yayma operatörü kullanılır: `setForm({ ...form, ad: e.target.value })`. Yalnızca `ad` alanını yeni değerle, diğer tüm alanları (`form` nesnesinin geri kalanını) olduğu gibi kopyalayan yeni bir nesne üretir — `ad`'ı doğrudan `form.ad = ...` ile değiştirmek, Hafta 4'te gördüğünüz gibi React'in referans karşılaştırmasını atlatır ve ekran güncellenmez.

5.2.3 `select` ve `checkbox`

Sınırlı bir seçenek kümesinden (koltuk sınıfı: standart/VIP) seçim için `<select>` kullanılır — tıpkı bir metin girdisi gibi `value`/ `onChange` çiftini izler. `checkbox` ise bir farkla çalışır: `value` yerine `checked` özniteliği, ve `e.target.value` yerine `e.target.checked` kullanılır — çünkü bir onay kutusunun "değeri" bir metin değil, bir doğru/YANLIŞtır.

> **İpucu**
>
> `checked` yerine yanlışlıkla `value` yazmak derleme hatası vermez — ama onay kutusu her zaman aynı görünümde kalır, tıklamalar hiçbir zaman `state`'e yansımaz. `checkbox` her zaman `checked`/`e.target.checked` çiftini kullanır, diğer girdi türleri ise `value`/`e.target.value` çiftini kullanır.

5.2.4 Basit doğrulama ve `onSubmit` + `preventDefault`

Bir `<form onSubmit={...}>` işleyicisinin ilk satırının neredeyse her zaman `e.preventDefault()` olması gerekir — aksi hâlde tarayıcının varsayılan davranışı (sayfayı yeniden yüklemek) devreye girer ve tüm React `state`'i sıfırlanır. Hafta 1 Ünite 4'ün HTTP istek/cevap bilgisini hatırlayın: normalde bir form gönderimi tarayıcıyı yeni bir HTTP isteğine yönlendirir; `preventDefault` bu isteği engeller — Hf.6'da gerçek bir istek göndermeye başlayana kadar form verisi yalnızca JavaScript içinde işlenir.

`preventDefault`'tan sonra, gönderim öncesi birkaç koşul kontrol edilir — ad boş mu, sepette en az bir koltuk seçili mi. Koşullardan biri sağlanmazsa `hata` state'i doldurulur ve fonksiyon erken `return` yapar; `Bilet[]` oluşturulmaz:

```
function submitEt(e: FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (form.ad.trim() === '') {
    setHata('Ad alanı boş olamaz.');
    return;
  }
  if (secilenKoltuklar.length === 0) {
    setHata('Sepette en az bir koltuk seçili olmalı.');
    return;
  }
```

Her iki koşul da sağlanırsa, sepetteki her koltuk için bir `Bilet` nesnesi `map` ile üretilir; bu diziler `onBiletlerOlustu` callback'iyle — Ünite 3'ün "durum kaldırma" konusunda ayrıntısını göreceğiniz bir kalıpla — üst bileşene bildirilir, sepet `sepetiTemizle()` ile boşaltılır ve form `BOS_FORM`'a sıfırlanır:

```
const satinAlmaZamani = new Date().toISOString();
const yeniBiletler: Bilet[] = secilenKoltuklar.map((koltuk) => ({
  id: sonrakiBiletId++,
  koltukId: koltuk.id,
  kullaniciAdi: form.ad,
  satinAlmaZamani,
  durum: 'olusturuldu',
}));

onBiletlerOlustu(yeniBiletler);
sepetiTemizle();
setForm(BOS_FORM);
setHata(null);
```

5.2.5 `BiletFormu` bileşeninin görünümü

Formun render'ı, ad alanı için bir metin girdisi, koltuk sınıfı için bir `select`, kampanya onayı için bir `checkbox` ve — varsa — `hata` mesajını gösterir. Ad alanı, bu ünitenin başında gördüğünüz kontrollü girdi döngüsünün somut hâlidir — `value` her zaman `form.ad`'tan okunur, her tuş vuruşu yayma operatörüyle yeni bir `form` nesnesi üretir:

```
<label>
  Ad Soyad
  <input
    type="text"
    value={form.ad}
    onChange={(e) => setForm({ ...form, ad: e.target.value })}
  />
</label>
```

Koltuk sınıfı ve kampanya onayı alanları yukarıda anlattığınız `select`/`checkbox` desenlerini birebir uygular; `hata` varsa altında bir uyarı, en altta gönderim düğmesi yer alır — formun tam render'ı `kod_ornekleri/hafta05/istemci/src/bilesenler/BiletFormu.tsx` dosyasındaki `bilet_form_render` kesitindedir.

`{hata && <p className="hata">{hata}</p>}` deseni, Hafta 4'ün `&&` operatörüyle koşullu render kalıbının burada da geçerli olduğunu gösterir — `hata` `null` olduğu sürece bu satır hiçbir şey basmaz.

3 • Katılım **⌨ Şimdi Siz Deneyin**

`BiletFormu`'nda ad alanını boş bırakıp gönderin, ekranda ne olduğunu gözlemleyin; sonra formu düzgün doldurup en az bir koltuk seçili durumdayken tekrar gönderin, sepetin ve koltukların durumunu inceleyin.

Geribildirim. Boş ad ile `hata` state'i "Ad alanı boş olamaz." mesajını gösterir; `submitEt` içindeki doğrulama erken `return` yapar, `Bilet[]` oluşturulmaz. Ad doldurulup en az bir koltuk seçiliyken gönderildiğinde ise seçili koltuklar `'dolu'` olur, sepet temizlenir ve form `BOS_FORM`'a döner.

Kendinizi Sınayın 5.2

Bir `<input>` öğesine `value={ad}` verip `onChange` vermezseniz ne olur?

5 • İzleme — Aktarım — Başka Nerede?

Bugün kurduğunuz tiplenmiş form state'i + doğrulama kalıbı, Hf.10'daki giriş/kayıt formlarında (kullanıcı adı/şifre alanları) aynı `value`/`onChange`/hata-mesajı desenle tekrar kullanılacak — bir form yazma alışkanlığınız artık tekrar tekrar uygulayacağınız bir kalıp hâline geldi.

5.3 Durum Kaldırma (Lifting State Up) ve Context

1 • Ön-öğretim — Neden Önemli?

`Koltuk` hâlâ kendi durumunu biliyor, ama `SepetOzeti` (kaç koltuk seçili, toplam tutar ne) ve `BiletFormu` (hangi koltuklar için bilet kesilecek) bu bilgiye ihtiyaç duyuyor — ve bu bilgi `Koltuk`'un içinde hapis. Bu ünite bu bilgiyi ortak, paylaşılan bir yere taşır.

1 • Ön-öğretim — Ön Bilgi

Hafta 4 Ünite 2'nin bileşen ağacı/tek yönlü veri akışı fikrini hatırlayın. Ayrıca Hafta 4'ün bilinçli olarak yanıtsız bıraktığı soruyu hatırlayın: `Koltuk`'un seçimi kendi içinde yaşıyordu — `App`'in ya da `KoltukPlani`'nın bundan haberi yoktu. Bu ünite tam olarak bu soruyu yanıtlar.

Hafta 4'ün sonunda bilinçli olarak yanıtsız bıraktığınız bir soru vardı: "`Koltuk`'un seçimi kendi içinde yaşıyor — `App`'te toplam kaç koltuk seçili, toplam tutar ne olduğunu nasıl görürüz?" Bu ünite tam olarak bu soruyla açılır: önce bu bilgiyi bir üst bileşene taşımanın (durum kaldırma) sınırlarını, sonra bunun yerine bir Context kurmanın nasıl daha ölçeklenebilir bir çözüm olduğunu görürsünüz.

5.3.1 Sorunun somutlaşması

`Koltuk` bileşeni kendi `durum` state'ini tutuyor (Hafta 4 Ünite 3), ama `SepetOzeti` ve `BiletFormu` bu bilgiye ihtiyaç duyuyor — ve bu bilgi `Koltuk`'un dışından görünmüyor. Hafta 4 Ünite 3'ün öğrettiği gibi bir `useState` çağrısı yalnızca tanımlandığı bileşene aittir; başka hiçbir bileşen bu değeri doğrudan göremez. Bu sınırı aşmanın iki yolu vardır: durumu bir üst bileşene taşımak (durum kaldırma), ya da bir Context kurmak. Önce ilkini, sonra ikincisini ve neden ikincisinin tercih edildiğini görün.

5.3.2 Durum kaldırma (*lifting state up*)

Durum kaldırma: Paylaşılması gereken bir durumu, ilgili bileşenlerin ortak üst bileşenine taşıma (*lifting state up*).

Durum kaldırma, birden fazla bileşenin paylaşması gereken bir durumu, bu bileşenlerin ortak en yakın üst bileşenine taşımaktır. Üst bileşen durumu tutar, alt bileşenlere hem güncel değeri (bir `props` olarak) hem de bu değeri değiştirmek için bir

callback fonksiyonu (yine bir `props` olarak) geçirir — alt bileşen durumu doğrudan değiştirmez, üst bileşene "şunu değiştir" diye seslenir (callback'i çağırır).

Analoji

Durum kaldırmayı iki masanın kendi ayrı not defteri tutması yerine ortak resepsiyona tek bir kayıt defteri koymaya benzetin: her iki masa da aynı deftere bakar, biri bir şey yazınca diğeri de görür; defter artık ikisinin de erişebildiği ortak bir yerdedir (üst bileşen).

5.3.3 Prop drilling sorunu

Durumu `App`'e kadar kaldırdığınızda, `App` ile durumu kullanan bileşen (`Koltuk`) arasında ara bileşenler (`KoltukPlani`) varsa, bu ara bileşenlerin `props`'una — kendileri o veriyle hiçbir iş yapmasa bile — o veriyi/callback'i sadece alt'a aktarmak için eklemeniz gerekir. Buna **Prop drilling** denir. Bileşen ağacı derinleştikçe bu "yalnızca aktarma amaçlı prop" sayısı katlanarak büyür, kod kırılganlaşır — bir ara bileşenin adı/imzası değişince zincir bozulur.

Prop drilling: Bir veriyi/callback'i, kullanmayan ara bileşenler üzerinden yalnızca aktarmak için prop olarak geçirme zorunluluğu (*prop drilling*).

Analoji

Prop drilling'i bir zarfı, ilgisi olmayan her ara kata elden uğratarak üst kata göndermeye benzetin: asansör (Context) yokken zarfı 5. kata göndermek için 2., 3. ve 4. kat sakinlerinin de zarfı eline alıp geçirmesi gerekir — hiçbiri zarfın içeriğiyle ilgilenmez, yalnızca elden ele taşırlar.

Şekil 5.3 bu iki yaklaşımı yan yana karşılaştırır: solda yalnızca `props`/callback ile taşımanın yol açtığı prop drilling, sağda bir Context ile aradaki bileşenlere uğramadan doğrudan erişim.

5.3.4 `createContext`/`useContext` ile Context

React bu sorunu çözmek için bir **bağlam (Context)** sunar: ağacın herhangi bir üst noktasında `<XContext.Provider value={...}>` ile "yayınlanan" ve ağacın herhangi bir alt noktasındaki bileşenin, aradaki bileşenlerin hiç haberi olmadan, `useContext(XContext)` ile doğrudan okuyabildiği bir veri kanalıdır — prop drilling'i "aradaki her katmana uğramadan" ortadan kaldırır. Context'i ağaca sağlayan bileşene bir **sağlayıcı (Provider)** denir.

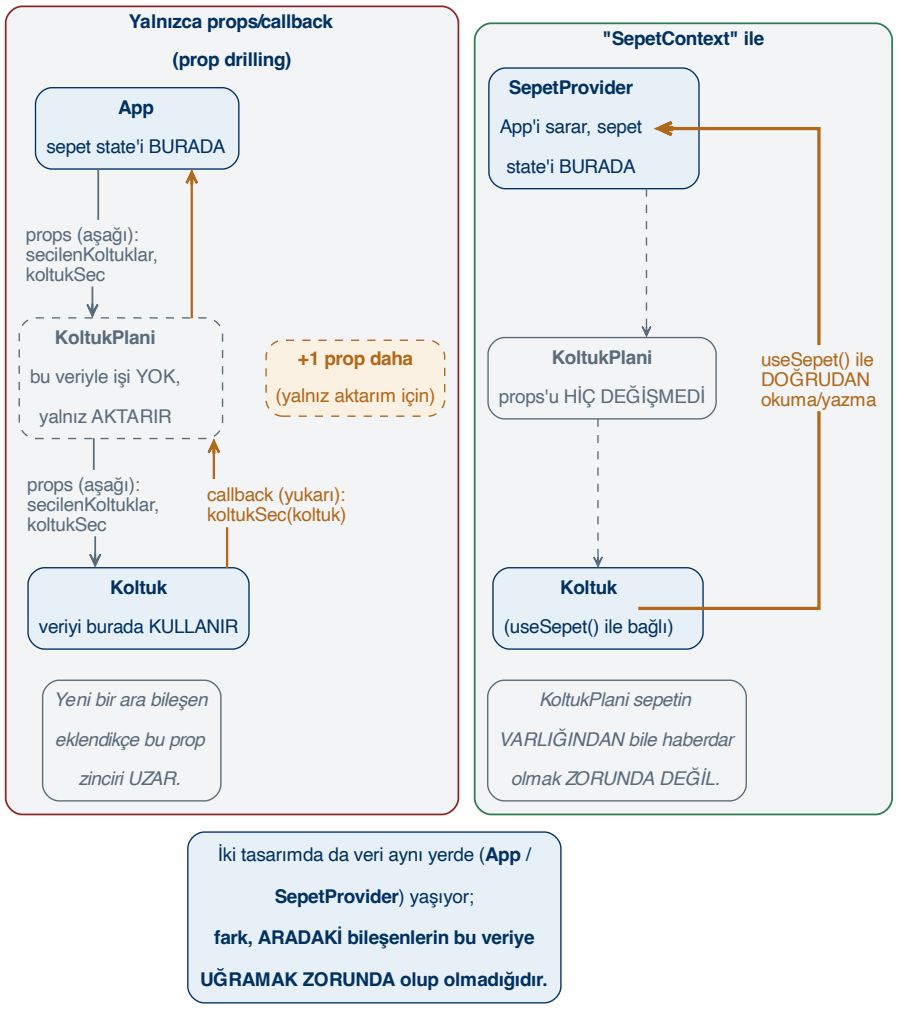
bağlam (Context): Ağacın herhangi bir üst noktasında sağlanan, herhangi bir alt noktadan aradaki bileşenlere uğramadan okunabilen veri kanalı (*context*).

Analoji

Context/Provider'ı bir binadaki ortak anons/santral sistemine benzetin: bina yönetimi (Provider) bir duyuruyu sisteme verir, hangi kattaki daire isterse (herhangi bir alt bileşen) doğrudan kendi hoparlöründen (`useContext`) dinler — duyuru kat kat elden ele dolaştırılmaz.

5.3.5 `SepetContext` tasarımı

Bu haftanın sepeti tam olarak bu kalıbı kullanır. Önce Context'in dışarıya açtığı veri ve işlevlerin şeklini bir arayüzle tanımlarsınız: seçili koltuklar, toplam tutar, bir koltuğu seçme/kaldırma fonksiyonu ve sepeti temizleme fonksiyonu:



Şekil 5.3: Sepet bilgisini paylaşmanın iki yolu: yalnızca props/callback ile prop drilling (solda) ve `SepetContext` ile aradaki bileşenlere uğramadan doğrudan erişim (sağda).

```ts
export interface SepetBaglami {
  secilenKoltuklar: Koltuk[];
  toplamTutar: number;
  koltukSec: (koltuk: Koltuk) => void;
  sepetiTemizle: () => void;
}
```

Context'in kendisi `createContext` ile oluşturulur; başlangıç değeri `null`'dur — bunun neden gerekli olduğunu Ünite 4'te göreceksiniz:

```ts
export const SepetContext = createContext<SepetBaglami | null>(null);
```

Son olarak `SepetProvider` bileşeni seçili koltukları `useState` ile tutar. `koltukSec`, koltuk zaten sepetteyse çıkarır, değilse ekler — değişmezlik ilkesiyle her zaman yeni bir dizi üretir:

```ts
export function SepetProvider({ biletFiyati, children }: SepetProviderProps)
    {
  const [secilenKoltuklar, setSecilenKoltuklar] = useState<Koltuk[]>([]);

  function koltukSec(koltuk: Koltuk) {
    setSecilenKoltuklar((onceki) =>
      onceki.some((k) => k.id === koltuk.id)
        ? onceki.filter((k) => k.id !== koltuk.id)
        : [...onceki, koltuk],
    );
  }
```

Sepeti boşaltan `sepetiTemizle`, dışa açılan değerleri (`secilenKoltuklar`, `toplamTutar`, `koltukSec`, `sepetiTemizle`) toplayan `deger` nesnesi ve bunu `children`'a (Hafta 4'ten tanıdık kompozisyon deseni) sağlayan `return` satırı şöyle tamamlanır:

```ts
  function sepetiTemizle() {
    setSecilenKoltuklar([]);
  }

  const deger: SepetBaglami = {
    secilenKoltuklar,
    toplamTutar: secilenKoltuklar.length * biletFiyati,
    koltukSec,
    sepetiTemizle,
  };

  return <SepetContext.Provider value={deger}>{children}</SepetContext.
      Provider>;
}
```

`toplamTutar` Ayrıca saklanmaz — her render'da `secilenKoltuklar`'dan (`secilenKoltuklar.length * biletFiyati`) yeniden hesaplanır. Bu, Ünite 1'in temizleme fonksiyonundaki mantığa benzer bir tasarım tercihidir: türetilebilen bir değeri ayrı bir `state`'te tutmak, iki değerin birbirinden sapması riskini doğurabilir; bunun yerine mevcut bilgiden her seferinde yeniden hesaplamak daha güvenlidir.

> **İpucu**
>
> Sepetteki eylemleri (`EKLE`/`CIKAR`/`TEMIZLE`) tek bir **reducer** fonksiyonunda da toplayabilirsiniz — `useReducer(sepetReducer, baslangicDurumu)`, `useState`'e alternatif bir durum yönetim biçimidir. Aşağıdaki `sepetReducer`, hangi eylem (`tur`) geldiyse durumun nasıl sonuçlanacağını tek bir yerde belir-

ler:

```
export type SepetEylemi =
  | { tur: 'EKLE'; koltuk: Koltuk }
  | { tur: 'CIKAR'; koltuk: Koltuk }
  | { tur: 'TEMIZLE' };

export function sepetReducer(durum: Koltuk[], eylem: SepetEylemi): Koltuk
    [] {
  switch (eylem.tur) {
    case 'EKLE': return [...durum, eylem.koltuk];
    case 'CIKAR': return durum.filter((k) => k.id !== eylem.koltuk.id);
    case 'TEMIZLE': return [];
  }
}
```

Bu fonksiyon `context/SepetContext.tsx` içinde dışa aktarılır, ama `SepetProvider`'a bağlanmaz — bu haftanın çalışan sepet mantığı sadelik için yukarıdaki `useState`'i kullanmaya devam eder. `useReducer` özellikle birden çok ilişkili eylem olduğunda — ör. eylemler birbirine bağımlı büyüdüğünde — tercih edilebilecek bir alternatiftir.

5.3.6 `Koltuk` bileşeni: yerel state'ten Context'e

`Koltuk` bileşeni artık kendi `useState`'ini tutmaz; koltuğun seçili olup olmadığını doğrudan `SepetContext`'ten okur:

```
const { secilenKoltuklar, koltukSec } = useSepet();
const seciliMi = secilenKoltuklar.some((k) => k.id === koltuk.id);
const durum = koltuk.durum === 'dolu' ? 'dolu' : seciliMi ? 'secili' : 'bos';

function tikla() {
  if (koltuk.durum === 'dolu') return;
  koltukSec(koltuk);
}
```

`secilenKoltuklar.some(...)` ile bu koltuğun sepette olup olmadığı kontrol edilir; `koltuk.durum === 'dolu'` ise (`veri.ts`'ten gelen kalıcı durum) `tikla` hiçbir şey yapmaz — `'dolu'` bir koltuk hiçbir zaman tıklanamaz. Görünüm kısmı Hafta 4'tekiyle aynı kalır:

```
return (
  <button
    type="button"
    className={`koltuk ${durum}`}
    onClick={tikla}
    disabled={durum === 'dolu'}
    aria-disabled={durum === 'dolu'}
  >
    {koltukEtiketi(koltuk)}
    {durum === 'dolu' && <span className="koltuk-rozet">Dolu</span>}
  </button>
);
```

Ortak Hata / Tuzak

`useSepet()`'i (bu fonksiyonun nereden geldiğini Ünite 4'te göreceksiniz) bir bileşenin içinde çağırmak, o bileşenin bir Context sağlayıcısının — `SepetProvider`'ın — altında durmasını gerektirir. `Koltuk` bileşenini yanlışlıkla `SepetProvider`'ın dışında bir yere yerleştirirseniz Context'ten okunan değer `null` olur — bu sorunun nasıl erken ve anlaşılır bir hatayla yakalandığını Ünite 4'te göreceksiniz.

3 • Katılım **Şimdi Siz Deneyin**

Kâğıt üzerinde (ya da bir yorum satırında) yalnızca `props`/callback ile `App`'teki sepet bilgisini `KoltukPlani` üzerinden `Koltuk`'a nasıl taşıyacağınızı tasarlayın; `KoltukPlani`'nın `props` tanımına kaç yeni alan eklemeniz gerektiğini sayın. Sonra `useSepet()`'i kullanarak aynı işi yapın ve `KoltukPlani`'nın `props` tanımının değişip değişmediğine bakın.

Geribildirim. Props yoluyla `KoltukPlani`'nın `props`'una en az iki yeni alan (`secilenKoltuklar`, `koltukSec`) eklemeniz gerekir — `KoltukPlani` bu veriyle kendi işinde hiçbir şey yapmaz, yalnız `Koltuk`'a aktarır (prop drilling). `useSepet()` ile `KoltukPlani`'nın `props`'u hiç değişmez — `Koltuk` sepeti doğrudan Context'ten okur.

Kendinizi Sınayın 5.3

`SepetContext` Olmasaydı, sepet bilgisini `App`'ten 2 seviye aşağıdaki `Koltuk`'a (`App` → `KoltukPlani` → `Koltuk`) yalnızca `props` ile taşımak isteseydiniz, `KoltukPlani`'nın `props` tanımında ne değişirdi, bu neden sorunlu olurdu?

5 • İzleme **Aktarım — Başka Nerede?**

Bu hafta kurduğunuz `SepetContext`, Hf.10'da bir `OturumContext` (giriş yapmış kullanıcı bilgisi, birçok sayfanın ihtiyaç duyacağı ortak bir durum) ile aynı kalıpla genişleyecek — "birden çok bileşenin ihtiyaç duyduğu ortak durum" sorusunun cevabı bundan sonra her yerde Context olacaktır.

5.4 Özel Hook'lar ve Kompozisyon

1 • Ön-öğretim **Neden Önemli?**

`useSepet` Olmasaydı, her bileşen (`Koltuk`, `SepetOzeti`, `BiletFormu`, `GeriSayim`) kendi içinde `useContext(SepetContext)` yazıp `null` kontrolünü tekrar tekrar yapmak zorunda kalırdı; `useGeriSayim`'in `setInterval` mantığı da başka bir yerde (ör. ileride farklı bir sayaç) gerekseydi kopyala-yapıştır yapardınız. Bu ünite tekrar eden mantığı tek bir yerde toplamayı öğretir.

1 • Ön-öğretim **Ön Bilgi**

Hafta 4 Ünite 1'in "neden bileşen" mantığını — tekrar kullanılabilir birim fikrini — hatırlayın; bu ünitede aynı fikir bileşenlerden fonksiyonlara (hook'lara) genişler. Ünite 1'in `useEffect`/temizleme bilgisini de hatırlayın — bu ünitedeki `useGeriSayim`'in içinde aynen kullanılır.

Ünite 1'de bir fonksiyon yazdınız (`useGeriSayim`) ve içinde `useState` ile `useEffect` kullandınız, ama bunu resmî olarak adlandırmadınız. Ünite 3'te `useSepet`'in adı geçti ama içini görmediniz. Bu ünite bu iki fonksiyonu resmî olarak tanımlar, bir bileşeni ne zaman ikiye bölmeniz gerektiğine dair pratik ölçütler verir ve haftanın tüm parçalarını `App`/`AppIcerik`'te birleştirerek Etkinlik/Bilet Sistemine ilk gerçek sepet katmanını tamamlar.

5.4.1 Özel hook nedir?

Özel hook: Adı `use` ile başlayan, içinde başka hook'lar çağıran, mantık paylaşan (JSX döndürmeyen) fonksiyon (*custom hook*).

Özel hook, adı `use` ile başlayan, içinde başka hook'lar (`useState`, `useEffect`, `useContext`) çağıran sıradan bir TypeScript fonksiyonudur. Bir bileşenin yapabileceği her şeyi yapabilir — state tutabilir, efekt çalıştırabilir — ama kendisi JSX döndürmez: yalnız mantık paylaşır, görünüm paylaşmaz (görünüm paylaşımı, birazdan göreceğiniz `children`'ın işidir).

Analoji

Özel hook'u birden fazla resepsiyon masasının aynı standart kayıt prosedürünü tekrar tekrar öğrenmesi yerine, prosedürü tek bir el kitabına yazıp herkesin ona başvurmasına benzetin: her masa (bileşen) aynı işi (sepeti okuma, geri sayım tutma) yeniden icat etmez, ortak el kitabını (hook'u) çağırır.

5.4.2 `useSepet()`: Context'i saran özel hook

`useSepet`, `useContext(SepetContext)`'i sarar:

```
export function useSepet() {
  const baglam = useContext(SepetContext);
  if (!baglam) {
    throw new Error('useSepet() yalnızca SepetProvider içinde kullanılabilir.'
        );
  }
  return baglam;
}
```

Context `null` dönerse — bir bileşen yanlışlıkla `SepetProvider` dışında kullanılmışsa — açık bir hata (`throw new Error(...)`) fırlatır. Bu sayede yanlış kullanım hemen, anlaşılır bir mesajla ortaya çıkar; "`toplamTutar` undefined" gibi belirsiz bir hatayla geç fark edilmez. Her bileşen (`Koltuk`, `SepetOzeti`, `BiletFormu`, `GeriSayim`) `useContext` + `null` kontrolünü tekrar tekrar yazmak yerine yalnızca `const { ... } = useSepet();` yazar.

Ortak Hata / Tuzak

`useSepet()` içindeki `if (!baglam) throw new Error(...)` satırını silerseniz ve bir bileşeni yanlışlıkla `SepetProvider`'ın dışında kullanırsanız,

`useContext(SepetContext)` `null` döner; bileşen `baglam.toplamTutar` gibi bir alana erişmeye çalıştığında "Cannot read properties of null" gibi belirsiz bir çalışma zamanı hatası alırsınız — hatanın gerçek nedeni (Provider eksikliği) mesajdan anlaşılmaz. Açık `throw` ile hata mesajı doğrudan sorunu ("`SepetProvider` içinde kullanılmalı") söyler; yanlış kullanım yeri hemen, net biçimde bulunur.

5.4.3 `useGeriSayim()`: artık resmî bir özel hook

Ünite 1'de yazdığınız `useGeriSayim` fonksiyonu, `use` ile başlayan adı ve içinde `useState`+`useEffect` çağırmasıyla zaten bir özel hookTU — yalnızca o ünitede bu terimi henüz kullanmamıştınız. `baslangicSaniye: number` parametresi alıp `{ kalanSaniye, bittiMi }` döndürür; aynı mantık başka bir yerde (ör. ileride farklı bir geri sayım) gerekseydi kopyala-yapıştır yerine bu hook yeniden çağrılırdı — `const {kalanSaniye, bittiMi} = useGeriSayim(60)` gibi, farklı bir başlangıç değeriyle bile.

5.4.4 Bileşen kompozisyonu kalıpları

bileşen kompozisyonu: Küçük bileşenleri birleştirerek (çoğunlukla `children` ile) daha büyük bir arayüz kurma yaklaşımı (*component composition*).

Hafta 4'te `Bolum` bileşeniyle `children` kullanarak bir bileşeni sarmalamayı görmüştünüz — bu, bir **bileşen kompozisyonu** biçimidir: küçük, tek işe odaklı bileşenleri birleştirerek daha büyük bir arayüz kurma yaklaşımı. İleri düzey bir başka kompozisyon kalıbının adı da **render props**'tır — bir bileşenin ne göstereceğini bir fonksiyon prop'u aracılığıyla çağırana bıraktığı bu desenin ayrıntısına bu hafta girilmez, yalnızca adını bilmeniz yeterlidir.

5.4.5 Bileşeni ne zaman bölmelisiniz?

Bir bileşeni ikiye bölmeniz gerektiğine dair üç pratik sinyal vardır:

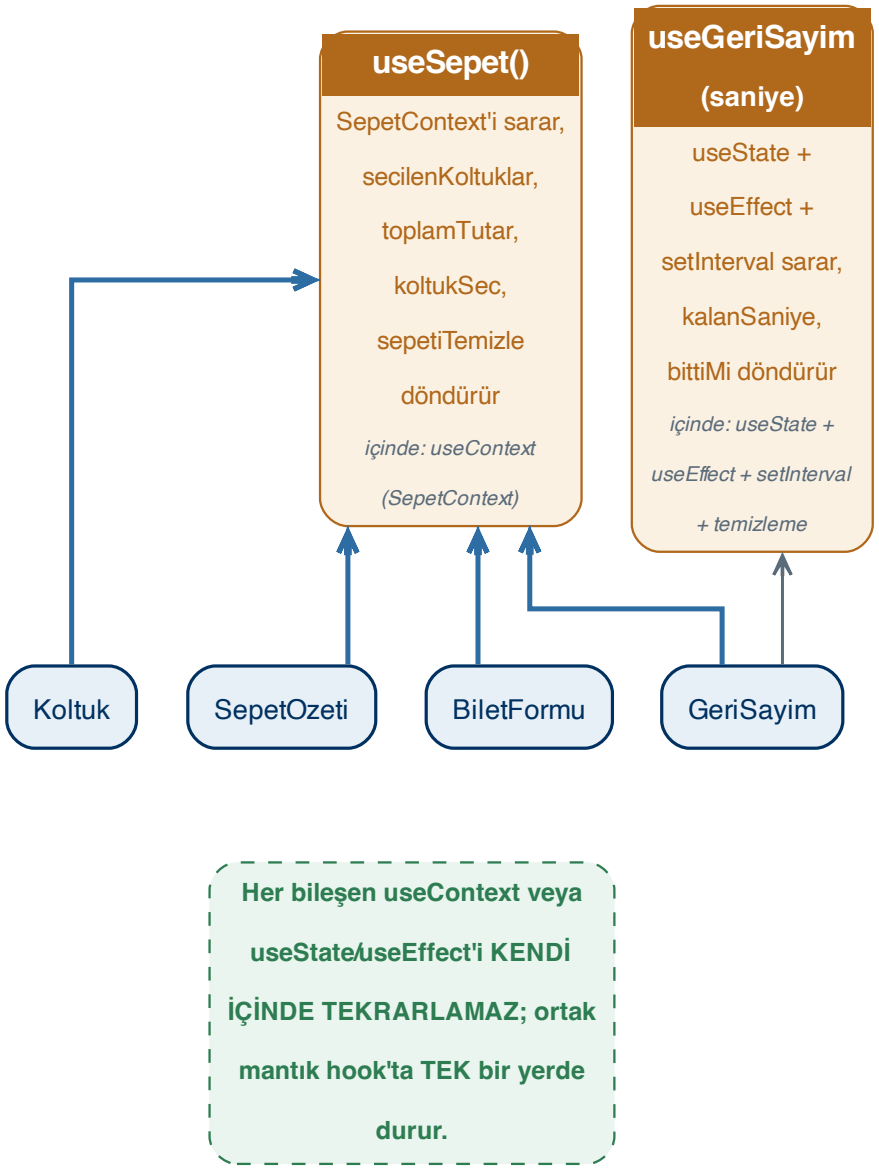
- Bileşen dosyası okumakta zorlandığınız kadar uzuyorsa.
- Aynı JSX/mantık parçası birden fazla yerde tekrarlanıyorsa.
- Bir bileşenin içinde birbiriyle ilgisiz iki sorumluluk (ör. hem form doğrulama hem sepet toplamı hesaplama) varsa.

Bu haftaki `BiletFormu` ile `SepetOzeti`'nin ayrı bileşenler olması, tam olarak üçüncü sinyalin bir uygulamasıdır — ikisi de sepet verisini kullanır, ama biri form doğrulamasından, diğeri yalnızca özet gösteriminden sorumludur.

5.4.6 Klasör düzeni

Bu hafta projeye üç yeni klasör türü eklendi: `src/bilesenler/` (JSX döndüren bileşenler), `src/hooks/` (özel hook'lar — JSX döndürmez) ve `src/context/` (Context tanımları ve Provider'lar). Bu ayrım, "bu dosya ne yapar" sorusunun dosya yolundan bile anlaşılmasını sağlar — bir `hooks/` dosyasını açtığınızda JSX beklemezsiniz, bir `context/` dosyasını açtığınızda bir `Provider` bileşeni beklersiniz.

Şekil 5.4, `useSepet` ve `useGeriSayim` hook'larının birden fazla, birbirinden bağımsız bileşen tarafından nasıl paylaşıldığını gösterir — `Koltuk`, `SepetOzeti` ve `BiletFormu` yalnızca `useSepet()`'i, `GeriSayim` ise hem `useSepet()`'i hem `useGeriSayim()`'i çağırır.



Şekil 5.4: `useSepet` ve `useGeriSayim` özel hook'larının `Koltuk`, `SepetOzeti`, `BiletFormu` ve `GeriSayim` bileşenleri tarafından paylaşılması.

5.4.7 Bu haftanın birleşimi: `App` ve `AppIcerik`

Artık tüm parçalar hazır: `SepetOzeti` yalnızca `useSepet()`'ten okur:

```
export function SepetOzeti() {
  const { secilenKoltuklar, toplamTutar } = useSepet();

  return (
    <div className="sepet-ozeti">
      <p>Seçili koltuk: {secilenKoltuklar.length}</p>
      <p>Toplam tutar: {toplamTutar} TL</p>
    </div>
  );
}
```

`GeriSayim` ise hem `useGeriSayim(120)`'u okur hem de süre bittiğinde (`bittiMi`) sepeti temizleyen ayrı bir efekt kurar — kullanıcı süresi içinde formu göndermezse sepet kendiliğinden boşalır:

```
const { kalanSaniye, bittiMi } = useGeriSayim(BASLANGIC_SANIYE);
const { sepetiTemizle } = useSepet();

useEffect(() => {
  if (bittiMi) sepetiTemizle();
}, [bittiMi]);
```

```
const dakika = Math.floor(kalanSaniye / 60);
const saniye = kalanSaniye % 60;
const goruntu = `${dakika}:${saniye.toString().padStart(2, '0')}`;

return (
  <div className="geri-sayim">
    {bittiMi ? (
      <p>Süre doldu, sepet temizlendi.</p>
    ) : (
      <p>Sepeti tamamlamak için kalan süre: {goruntu}</p>
    )}
  </div>
);
```

`SepetOzeti` ve `GeriSayim` için eklenen kurallar `style.css`'e şöyle eklenir — ikisi de düzenini `display: flex` ile kurar, rengini KTÜ paletinin bir değişkeniyle (`--renk-acikmavi`, `--renk-kehribar`) belirler:

```
.sepet-ozeti {
  display: flex;
  gap: 1.5rem;
  padding: 0.75rem 1rem;
  margin: 1rem 0;
  background-color: var(--renk-acikmavi);
  border-radius: 0.5rem;
  font-weight: bold;
}

.geri-sayim {
  margin: 0.5rem 0 1rem;
  color: var(--renk-kehribar);
  font-weight: bold;
}
```

`BiletFormu`'nun `hata` sınıfı da aynı dosyada, mesajı `--renk-bordo` ile vurgular:

```
.bilet-formu .hata {
  color: var(--renk-bordo);
  font-weight: bold;
}
```

Formun geri kalan stilleri (`.bilet-formu`, girdi/`select` alanları, gönderim düğmesi ve `.bilet-ozet`) aynı `kod_ornekleri/hafta05/istemci/src/style.css` dosyasındadır.

Son olarak `App` bileşeni artık yalnızca `SepetProvider`'ı kurar — hangi etkinliğin bilet fiyatını kullanacağını belirler:

```
export function App() {
  return (
    <SepetProvider biletFiyati={AKTIF_ETKINLIK.biletFiyati}>
      <AppIcerik />
    </SepetProvider>
  );
}
```

Asıl sayfa içeriği `AppIcerik` bileşenindedir; `AppIcerik`'in `SepetProvider`'ın içinde (bir alt bileşen olarak) durması gerektiğini unutmayın — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz, yalnızca alt'ları okuyabilir. `AppIcerik` kendi `koltuklar`/`biletler` state'ini tutar (sunucu yok, bu yüzden kalıcılık yok):

```
const [koltuklar, setKoltuklar] = useState<KoltukTipi[]>(ilkKoltuklar);
const [biletler, setBiletler] = useState<Bilet[]>([]);
const { secilenKoltuklar } = useSepet();
```

`BiletFormu`'ndan gelen `onBiletlerOlustu` callback'i — Ünite 3'ün durum kaldırma kalıbının bir örneği — yeni biletleri listeye ekler ve ilgili koltukları `'dolu'` durumuna geçirir:

```
function biletleriOlustur(yeniBiletler: Bilet[]) {
  setBiletler((onceki) => [...onceki, ...yeniBiletler]);

  const doluKoltukIdleri = new Set(yeniBiletler.map((bilet) => bilet.koltukId
      ));
  setKoltuklar((onceki) =>
    onceki.map((koltuk) =>
      doluKoltukIdleri.has(koltuk.id) ? { ...koltuk, durum: 'dolu' } : koltuk,
    ),
  );
}
```

Ve son olarak tüm bileşenler `AppIcerik`'in render'ında bir araya gelir:

```
return (
  <>
    <Bolum baslik="Etkinlikler">
      <EtkinlikListesi etkinlikler={etkinlikler} />
    </Bolum>
    <Bolum baslik="Koltuk Planı">
      <KoltukPlani koltuklar={koltuklar} />
    </Bolum>
    <Bolum baslik="Sepet">
      <SepetOzeti />
      <GeriSayim />
      <BiletFormu onBiletlerOlustu={biletleriOlustur} />
      {biletler.length > 0 && (
        <p className="bilet-ozet">Oluşturulan bilet sayısı: {biletler.length}</
            p>
```

```
      )}
    </Bolum>
  </>
);
```

Bu tek dosya, bu haftanın dört ünitesinde öğrendiğiniz her şeyi bir araya getirir: `useEffect` ile belge başlığı efekti (Ünite 1), kontrollü `BiletFormu` (Ünite 2), `SepetProvider` ile `Koltuk`'un Context'ten okuması (Ünite 3) ve `useSepet` ile `useGeriSayim` özel hook'ları (bu ünite). `npm run dev` çalıştırdığınızda artık "Sepet" başlıklı üçüncü bir bölüm görürsünüz: seçili koltuk sayısı ve toplam tutarı gösteren `SepetOzeti`, 120 saniyeden geri sayan `GeriSayim` ve `BiletFormu`. Geçerli bir gönderimde seçili koltuklar `'dolu'` olur, sepet ve form temizlenir.

3 • Katılım **⌨ Şimdi Siz Deneyin**

`useSepet()` çağrısını geçici olarak silip yerine doğrudan `useContext(SepetContext)` yazarak `null` kontrolünü `Koltuk`, `SepetOzeti` ve `BiletFormu`'nun üçünde de tekrarlayın; sonra hepsini `useSepet()`'e geri döndürüp kod satırı sayısını karşılaştırın.

Geribildirim. Doğrudan kullanım aynı 2-3 satırlık `null` kontrolünü üç ayrı yerde tekrarlatır; `useSepet()` bu kontrolü tek bir yerde yapar, bileşenler yalnızca `const { ... } = useSepet();` yazar — "bir kez yaz, çok yerde kullan" ilkesinin hook'lara uygulanmış hâlidir.

Kendinizi Sınayın 5.4

`useSepet()` hook'u içindeki `if (!baglam) throw new Error(...)` satırını silerseniz ve bir bileşeni yanlışlıkla `SepetProvider`'ın dışında kullanırsanız ne olur, bu neden daha kötü bir hata deneyimidir?

5 • İzleme **Aktarım — Başka Nerede?**

Bugün kurduğunuz `useSepet`/`useGeriSayim` kalıbı, Hf.7'de kuracağınız `useEtkinlikler()` (API'den veri çeken bir özel hook) ve Hf.10'un `useOturum()` hook'larının doğrudan atasıdır — özel hook yazma becerisi dönem boyunca tekrar tekrar kullanılacaktır. Bugünkü birleşimle Hafta 4'te `Koltuk` içinde hapis kalan koltuk seçimi artık `SepetContext` üzerinden `App` düzeyinde paylaşılıyor — Etkinlik/Bilet Sisteminin ön yüzü ilk gerçek sepet katmanına kavuştu.

Bölüm Özeti

Özet

Bu hafta, Hafta 4'ün "`Koltuk`'un seçimi kendi içinde yaşıyor, geri kalan hiçbir bileşen bunu görmüyor" sınırını ortadan kaldırdınız. İlk ünitede bir bileşenin render işinin dışına nasıl çıktığını — `useEffect`'i, bağımlılık dizisinin üç hâlini ve temizleme fonksiyonunu — belge başlığı güncelleyen ve `setInterval` tabanlı bir geri sayım örneğiyle öğrendiniz. İkinci ünitede Hafta 3'ün React'ten habersiz HTML formunu, `value`/`onChange` ile kontrollü bir

girdiye, tiplenmiş bir `BiletFormVeri` state'ine, `select`/ `checkbox` kullanımına ve `onSubmit`+`preventDefault` ile doğrulanan bir gönderime dönüştürdünüz. Üçüncü ünitede durum kaldırmanın sınırlarını (prop drilling) gördünüz ve `createContext`/ `useContext` ile bir `SepetContext` kurarak `Koltuk`, `SepetOzeti` ve `BiletFormu`'nun aynı sepeti paylaşmasını sağladınız. Son ünitede bu tekrar eden mantığı `useSepet` ve `useGeriSayim` özel hook'larında topladınız, bir bileşeni ne zaman bölmeniz gerektiğine dair ölçütleri gördünüz ve tüm bu parçaları `App`/`AppIcerik`'te birleştirerek Etkinlik/Bilet Sistemine ilk gerçek sepet katmanını — koltuk seçiminden `Bilet[]` oluşumuna kadar — eklediniz. Gelecek hafta bu `Bilet[]` nesnelerinin `App`'in yerel belleğinin dışına, gerçek bir sunucuya (FastAPI) taşınmasını göreceksiniz.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — bağımlılık dizisi ve sonsuz döngü). `useEffect(() => { ... }, [])` efekti yalnız ilk render'dan sonra bir kez çalıştırır — bağımlılık dizisi boş olduğu için React bu efektin "hiçbir bağımlılığı değişmedi" varsayımıyla bir daha çalıştırmaz. İkinci biçimde (`useEffect(() => { ... })`) ikinci argüman tamamen atlanmıştır — bu durumda efekt her render'dan sonra çalışır. Efekt içinde bir `setState` çağrısı varsa, bu çağrı yeni bir render'ı tetikler; o render tamamlandığında efekt tekrar çalışır, efekt içindeki `setState` yeniden bir render tetikler — bu döngü hiçbir dışsal koşul araya girmeden sonsuza dek sürer.

Soru (Ünite 2 — `value` verip `onChange` vermemek). React konsola "salt okunur alan" (*read-only field*) uyarısı basar; girdi kullanıcı için adeta yazılamaz hâle gelir. Bunun nedeni, `value`'nun her render'da aynı `state`'ten okunmasıdır — kullanıcı bir tuşa bassa bile, DOM'un kendi iç değeri her tuş vuruşundan hemen sonraki render'da bu `state`'e geri zorlanır. `state`'i güncelleyecek tek yol (`onChange`) tanımlı olmadığından, kullanıcı yazdığı harfi ekranda asla göremez.

Soru (Ünite 3 — `SepetContext` olmadan props ile taşıma). `KoltukPlani`'nın `props` tanımına `secilenKoltuklar` ve `koltukSec` gibi alanlar eklemeniz gerekirdi — kendisi bu veriyle hiçbir iş yapmadığı hâlde — çünkü bu veri yalnızca `KoltukPlani`'nın üzerinden geçip `Koltuk`'a ulaşabilir (prop drilling). Bu sorunludur çünkü ağaç derinleştikçe ya da yeni bir ara bileşen eklendikçe bu "yalnız aktarma amaçlı" prop sayısı katlanarak büyür ve `KoltukPlani`'nın imzası, kendi işiyle ilgisiz bir sebeple değişmiş olur — bileşenin kendi sorumluluğuyla alakasız bir kırılganlık kaynağı oluşturur.

Soru (Ünite 4 — `useSepet`'teki `throw`'u silmek). `useContext(SepetContext)` `null` döner; bileşen `baglam.toplamTutar` gibi bir alana erişmeye çalıştığında "Cannot read properties of null" gibi belirsiz bir çalışma zamanı hatası alırsınız — hatanın gerçek nedeni (bileşenin `SepetProvider` dışında kullanılmış olması) hata mesajından hiçbir şekilde anlaşılmaz, geliştirici hatayı bulana kadar kod içinde geriye doğru iz sürmek zorunda kalır. Açık `throw` ile hata mesajı doğrudan sorunu ("`useSepet()` yalnızca `SepetProvider` içinde kullanılabilir") söyler; yanlış kullanım yeri hemen, net biçimde bulunur — bu, kötü bir hata deneyimi (belirsiz, geç fark edilen) ile iyi bir hata deneyimi (açık, erken fark edilen) arasındaki farkın somut bir örneğidir.